

CSC3002F - Operating Systems II

Scheduling Assignment

Connor Morrison (MRRCON006)

19th May 2026

Introduction

Allegra the barman is an abstraction of the low-level workings of a CPU. The study of CPU scheduling algorithms can be likened much to the analogy of a barman serving drinks at a bar, where each patron (customer) is a process and each of their drink orders is a CPU burst. In this analogy, Allegra the barman acts as the CPU and her method of serving patrons is a scheduling algorithm. This provides us with a real world example of what CPU scheduling algorithms do and what these algorithms aim to achieve.

The CPU scheduling algorithms that will be tested within this analogy are First-Come First-Serve (FCFS), Shortest Job First (SJF), Priority, and Multilevel Feedback Queue (MLFQ). We are provided the information that none of the algorithms use pre-emption at the level of the individual drink order.

We will evaluate the overall success of the algorithms by comparing their performances with respect to the response and waiting times (both per order, and the total per patron), and turnaround time, as well as throughput, predictability, fairness, and possibility of starvation.

The goal of this assignment is to determine the CPU scheduling algorithm that, when applied to the bar setting, is to satisfy the bar owner.

In order to achieve maximum satisfaction, we need to serve as many customers in the shortest time frame (throughput), while at the same time, ensuring that all customers feel fairly treated such that they will, on average, have the best experience. Taking both of these things into account will help maximise nightly profit, while encouraging consequent visits, hopefully providing longer term success to the bar.

Methods

One of the most important steps in the experimental process is creating valid and reliable data in a repeatable way. This process can be broken down into steps to first generate the data, and second validate the data.

Method of data generation

To generate reliable and repeatable data we need to define, in context, what this would mean. For our results to be reliable we must first ensure that we fairly test each algorithm. We can do this by having simulations for each algorithm run with the same `seed`, `sleep_time`, `total_patrons` configuration.

`sleep_time` describes to the Java program the resting time of the barman before processing the next order, which in this scenario is unnecessary, so we set it to 0.

We need a range of different `total_patrons` to test these algorithms under different workloads. The values of 5, 20, and 50 have been chosen for this experiment.

According to the Central Limit Theorem, the minimum number of unique simulations required to model their distributions is 30. Meaning that we will need to test every scheduling algorithm 30 times for each value of `total_patrons`. To do this we can loop through seeds 1 to 30 for each `scheduling_algorithm`, `total_patrons` pairing.

We created a bash script Figure 1 to run through this repeated simulation process.

We need to also ensure that the generated data is in a suitable format for analysing. This was done by completing the `recordCompletedOrder` function in `Barman.java` to print all of the *per-order metrics* that the `barman` class provided in the format “*patron,drink,priority,queue_level,seq_no,service_start_time,arrival_time,completion_time,waiting_time,imbibing_time,turnaround_time,enqueue_time,execution_time,response_time*.” This output prints this for every completed drink order into the error console. We retrieve only the error console output of the application by calling `2>&1 ./dev/null` after the call of the `make` function.

But before we save the metrics in a `.csv` file, we need to attach the individual simulation configuration “`total_patrons,schedalg,seed,`” to the start of each line in the error console. This is done by `sed "s/^/$patrons,$schedalg,$seed,/"`. The final outputs are appended to `simulation.csv`.

By running our bash script, our generation of repeatable and reliable data is complete.

```

simulation.sh
1  echo
   "total_patrons,schedalg,seed,patron,drink,priority,queue_level,seq_no,service_start_time,arrival_time
   ,completion_time,waiting_time,imbibing_time,turnaround_time,enqueue_time,execution_time,response_time
   " > simulation_results.csv
2  for patrons in 5 20 50
3  do
4      for seed in {1..30}
5      do
6          for schedalg in 0 1 2 3
7          do
8              #args format ARGS="patrons scheduling_algorithm 0 seed"
9              make --silent run ARGS="$patrons $schedalg 0 $seed" 2>&1 >/dev/null | sed
               "s/^/$patrons,$schedalg,$seed,/" >> simulation_results.csv
10             done
11         done
12     done
13 # Outputs the results into simulation_results.csv in the format:
14 #
   patron,drink,priority,queue_level,seq_no,service_start_time,arrival_time,completion_time,waiting_time
   ,imbibing_time,turnaround_time,enqueue_time,execution_time,response_time

```

Figure 1: Screenshot of Bash Simulation Script

Method of data validation

The exact code/methods used to obtain results in the section can be found in *MRRCON006 Final Report.qmd*.

To validate the data and code for the analysis of the algorithms, we need to ensure that

(a) the data is complete, (b) the data follows the basic CPU scheduling rules, and (c) the data produced identical workloads for each algorithm across seeds.

(a) Data completeness

We check that the correct amount of simulations was run and that there were no missing values

Validation: Seeds per workload/algorithm combination:

total_patrons	schedalg	seeds_count
5	FCFS	30
5	SJF	30
5	Priority	30
5	MLFQ	30
20	FCFS	30
20	SJF	30

total_patrons	schedalg	seeds_count
20	Priority	30
20	MLFQ	30
50	FCFS	30
50	SJF	30
50	Priority	30
50	MLFQ	30

Number of missing values: 0

Which correctly matches what we expect to see.

(b) Code and data is logically correct

To test this we check whether all $Turnaround \geq Waiting + Execution$, and $Response \leq Waiting$. If there are 0 logic errors, then the code used and data generated is valid.

0 exceptions, Validation Passed: Turnaround times are logically consistent.

0 exceptions, Validation Passed: Response time is always $\leq$ total waiting time.

(c) Are the workloads identical across algorithms for the same seed

We tested whether the each algorithm had and identical total and type of drinks for the same seed, total_patrons pairing. This confirms that our simulation script provides valid, repeatable and reliable data.

Passed! The workload produced was identical across algorithms for a given seed

Results and Analysis

In this analysis, larger drinks will be used in place of drinks with longer preparation time, and smaller drinks will be used in place of drinks with shorter preparation time.

Per-Order metrics

The per-order metrics help us analyse the overall performance and efficiency the CPU scheduling algorithms (Allegra the bartender).

Response and waiting time

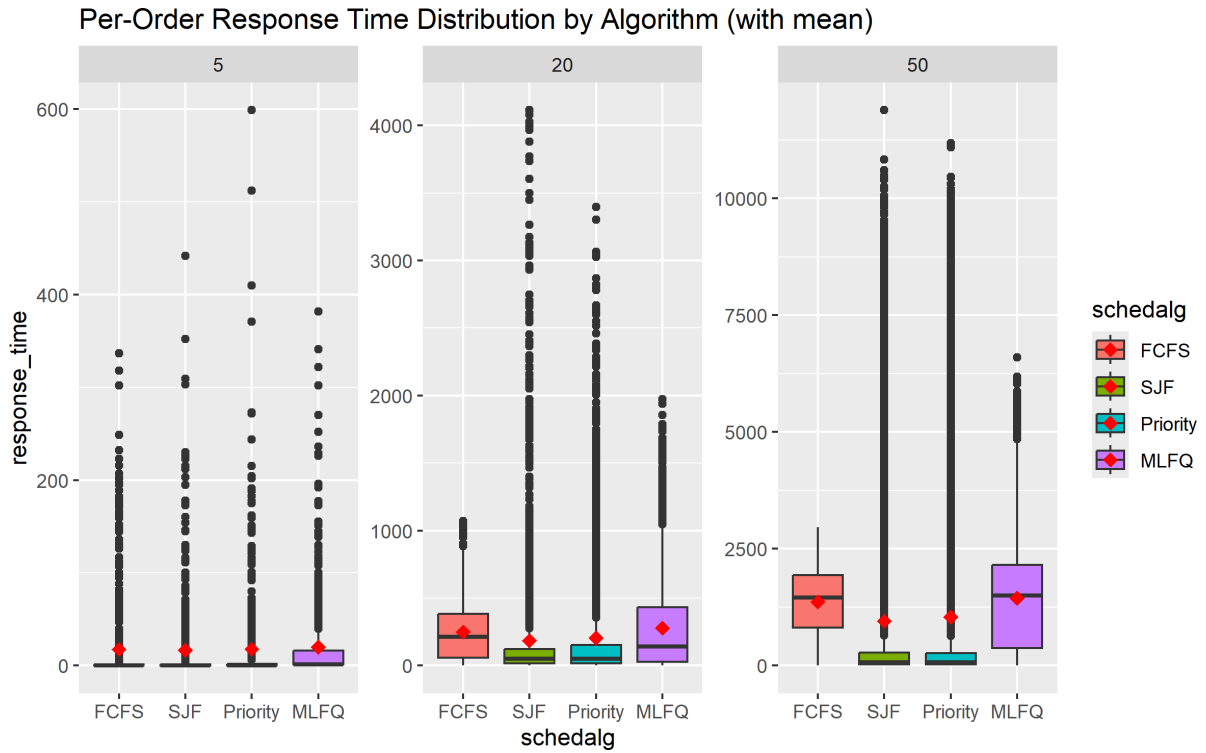


Figure 2: Boxplots of Per-Order Response Times

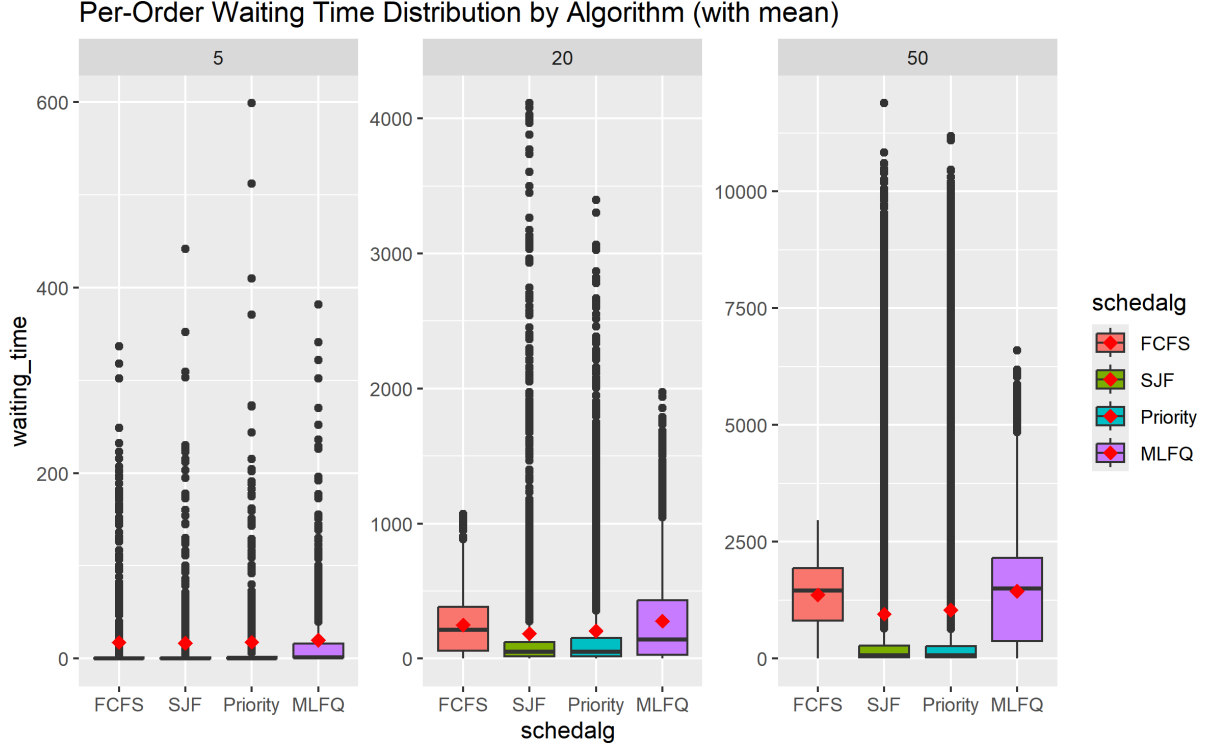


Figure 3: Boxplots of Per-Order Waiting Times

When comparing the Per-Order Response (Figure 2) and Per-Order Waiting (Figure 3), we observe that the plots are effectively identical, so the results and analysis of per-order response and waiting times will be done in unison. This is an expected behaviour and is further evidence that the data generated is valid.

Anaylsing the box-plots, we see that at `total_patrons = 5`, the distributions for the response/waiting times are almost identical. This is noticeable by the mean and medians all being in-line with similar amounts of outliers. This shows us that at a low workload, the effect of the individual scheduling algorithm is unnoticeable.

We can see that between `total_patrons` equals 20 and 50, the response/waiting time distributions of each algorithm are almost identical in shape, with the only difference being an increase in magnitude.

Both FCFS and MLFQ have means and medians that are very close together, this implies that the waiting/response times are more evenly distributed on a per-order basis.

For FCFS, we can see that there are little to no outliers, with a lower concentration of short waiting/response times. This is an expected result as it is an inherently fair algorithm.

MLFQ has few outliers with a wider interval boxplot graph. Both SJF and Priority have plenty of outliers, but also have very high concentrations of very low waiting times. This is the expected result when considering the nature of these algorithms.

SJF consistently has the shortest average waiting time, with Priority following closely behind and both FCFS and MLFQ having relatively much longer average waiting/response times.

Per-Patron metrics

These metrics show us the overall times that the patrons spend at the bar per night. Per-Patron metrics help us analyse the customer's satisfaction rate.

Response and waiting time

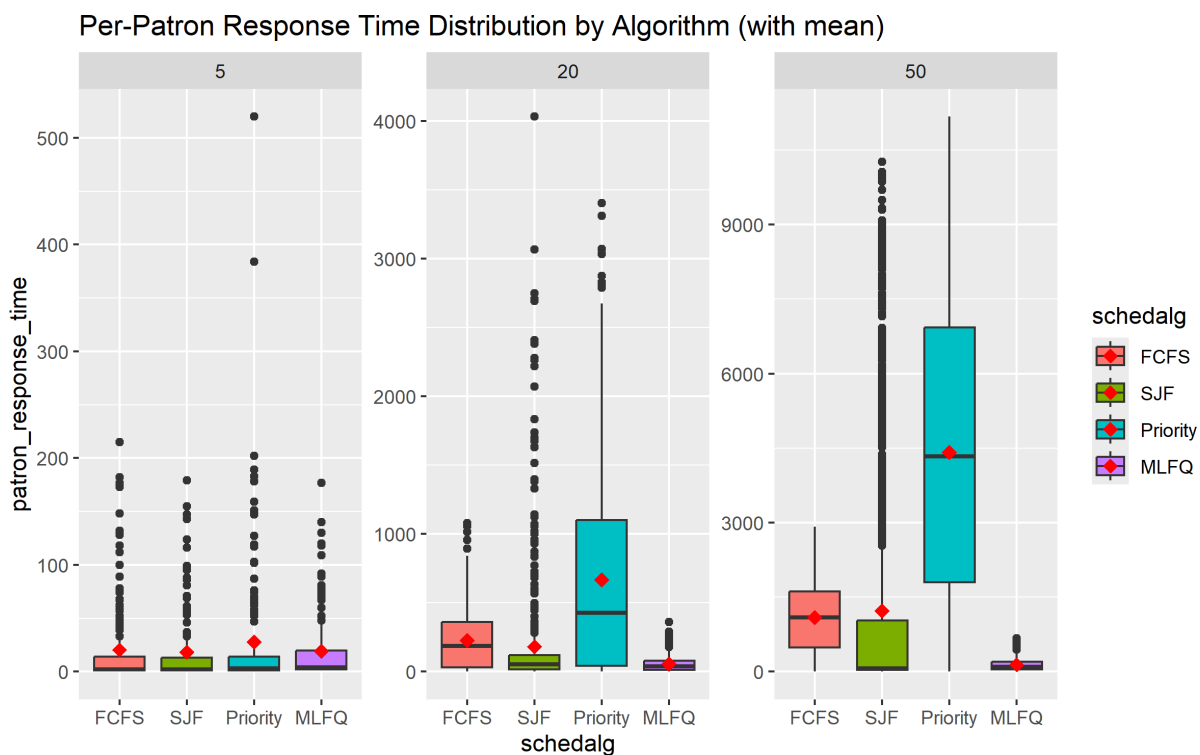


Figure 4: Boxplots of Per-Patron Response Times

The per-patron response time distributions exhibit very similar properties to the per-order response times, especially for `total_patrons = 5`.

As for 20 and 50:

MLFQ consistently displays the lowest median and mean values with the highest concentration of low per-patron response times, with few small outliers. FCFS and SJF have similar means, with SJF having a median close to MLFQ. Saying this, SJF has plenty of longer outliers, whereas, FCFS has a very uniform distribution with a much lower max per-patron response time.

Priority has little outliers, but has by far the longest mean and median response time. It also has a long max response time and a very low concentration of low response times. This shows us that it consistently treats patrons the same way based on their priority.

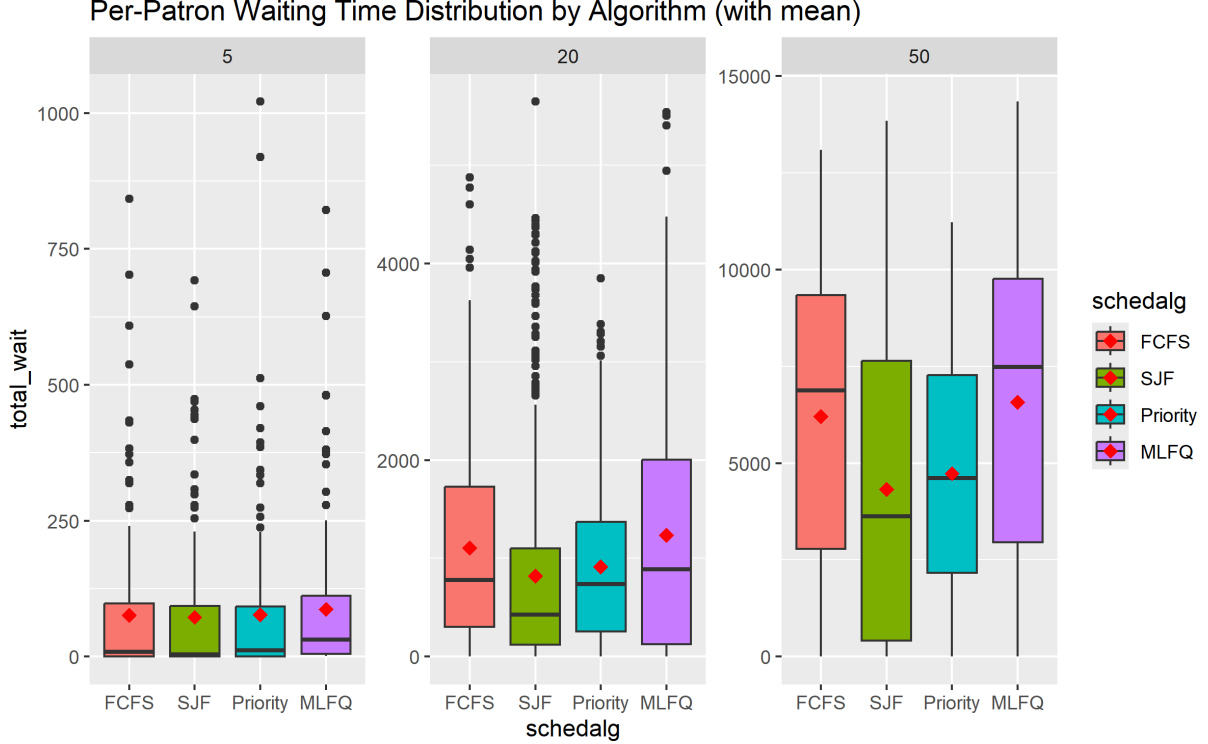


Figure 5: Boxplots of Per-Patron Waiting Times

Similar to the per-order waiting times, we see that when `total_patrons` = 5, the choice of algorithm does not matter.

But for the medium and high workloads, we observe that both graphs have a very similar shape. We see that there are few outliers for the medium workload and no outliers for the high workload. This shows us that on a patron-level the waiting time that one would experience is very predictable and consistent.

We observe that the mean and median per-patron waiting times are lowest for SJF, then Priority, then decently higher for FCFS and MLFQ. This is very in line with what we expected for the average waiting times of these algorithms.

Turnaround time

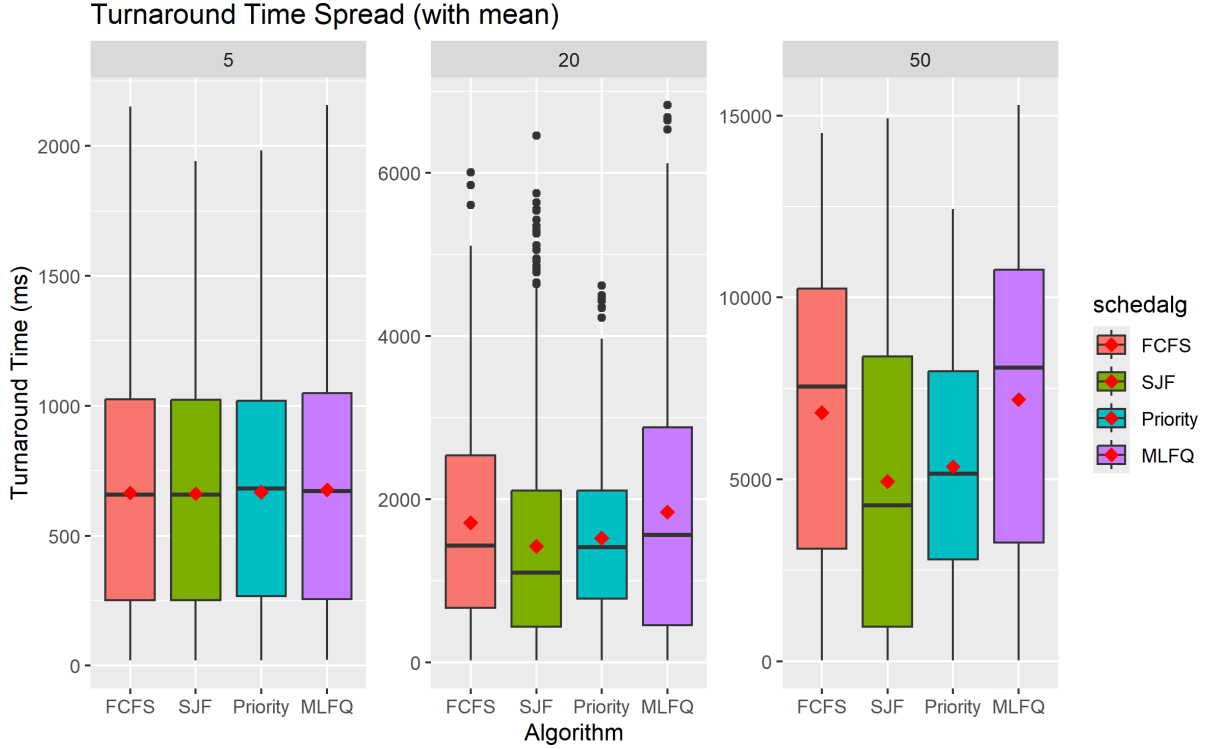


Figure 6: Boxplots of Turnaround Times

Similar to the per-patron waiting times, we see that when `total_patrons` = 5, the choice of algorithm does not matter.

But for the medium and high workloads, we observe that both graphs have a very similar shape. We see that there are few outliers for the medium workload and no outliers for the high workload. This shows us that on a patron-level the turnaround time that one would experience is very predictable and consistent.

We observe that the mean and median per-patron turnaround times are lowest for SJF, then Priority, then decently higher for FCFS and MLFQ. This is very in line with what we expected for the average turnaround times of these algorithms.

These results much reflect the per-patron waiting times. This shows us that the preparation time of the drinks has no real effect on the turnaround times. This means that the time a person spends at the bar is predominantly dependent on the waiting time at the bar.

Throughput

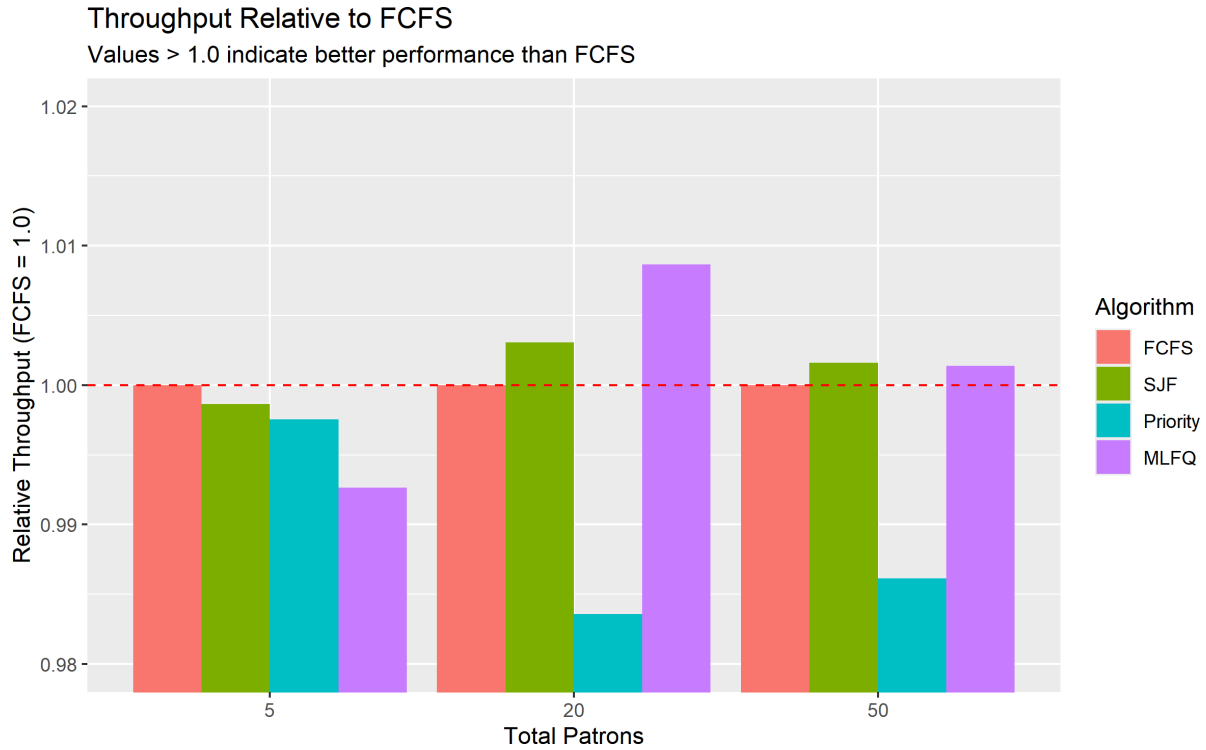


Figure 7: Relative Throughputs compared to FCFS

This graph shows us the average throughput of each algorithm relative to FCFS for different workloads.

For a workload of 5 patrons, we see that FCFS is by far the most efficient algorithm. For larger workloads, we can see that Priority has by far the worst efficiency. For a workload of 20 patrons, we see that MLFQ is by far the most efficient algorithm. And for a workload of 50 patrons, we can see that SJF and MLQF have very similar overall efficiency.

Fairness

Priority as evident from Figure 4: The results show us that priority is “fair to the system” showing us that for each priority level, the response times are very consistent. When looking at Figure 2 we can also see that although the Priority algorithm is “fair to the system”, it is unfair when looking at the per-order response and waiting times. i.e. if you have high priority, you will have a low response time.

Similarly to Priority, SJF is an algorithm that is “fair to the system,” Figure 5, but when looking at Figure 4 and Figure 2 we can see that SJF is unfair to larger drink orders and is always biased towards smaller drinks first. This means that SJF is unfair on a per-order level.

Shown in Figure 4 and Figure 2, we can see that both FCFS and MLFQ are very fair algorithms (little to no outliers) with FCFS being the most fair and MLQF being just slightly less fair.

Predictability

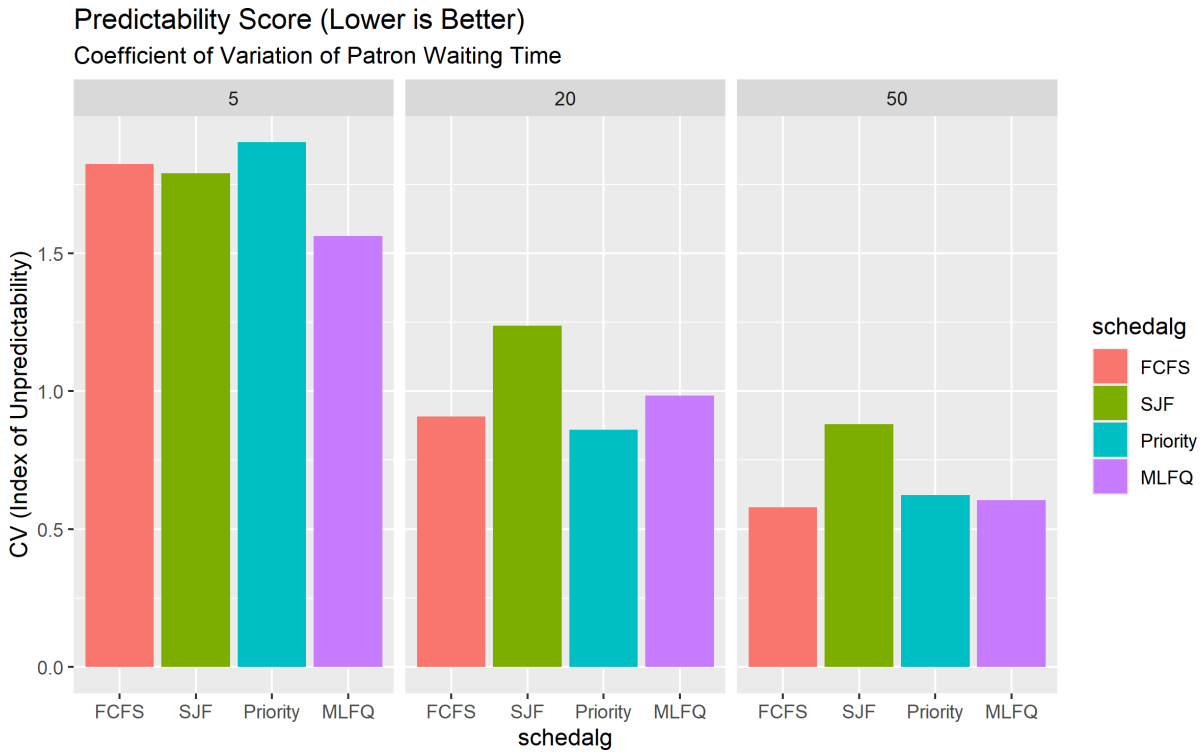


Figure 8: Predictability Score for Patron Waiting Time

Figure 8 substantiates our argument that the algorithm used with small workload is not important and quite erratic, as they all score poorly on the predictability score.

We can see that for higher workloads, SJF is very unpredictable which makes sense as if you order a large drink, you have no in cling for how many people you will end up waiting behind (unpredictably waiting time).

We also see that FCFS, MLFQ, and Priority have very similar predictability scores for larger workloads, meaning that a patron can better predict how long they will be waiting in line for

throughout the night. When we combine these results with the results shown in Figure 3 we can see it is much harder to predict the per-order waiting time for the Priority. They also show that FCFS is the easiest to predict with MLFQ following closely behind. These results all make sense when you relate back to the fairness of the algorithm.

Possibility of Starvation

As observed in the per-order waiting times, Figure 3, we see that for both SJF and Priority, there are many outliers with extremely large waiting times and the distributions are heavily skewed. This is evidence of starvation as for this to happen, some orders must be neglected for long periods of time.

In the case of SJF, the bartender will always serve patrons who order smaller drinks first, so when a patron orders a large drink they keep getting pushed to the back of the line until all patrons who order smaller drinks are served first. This means that SJF is highly prone to the possibility of starvation.

In the case of Priority, the bartender will always serve patrons with a higher priority, so when a patron has a lower priority, they will be neglected until all patrons of higher priority are served. This means that Priority is also highly prone to the possibility of starvation.

In Figure 3, MLFQ has a low amount of outliers. This means that there is still a low chance of starvation. This makes sense as there is some tiering to the algorithm so it is not completely fair.

It is noticed in Figure 3 that FCFS has no outliers for waiting times in a high workload scenario. This means that FCFS has 0 chance of starvation which logically makes sense. Since, as soon as you join the queue to be served you have a guarantee that you will be served in the same order.

Conclusions and Recommendations

There are many factors that influence the decision of the best scheduling algorithm to satisfy a bar owner. On one hand, the algorithm with the highest throughput and lowest average waiting times (SJF) will result in the most nightly profit; but this will likely scare many patrons from returning if they felt like they were treated unfairly, and hence lowering the amount of recurring customers and long-term profit for the bar. So we want to find the algorithm that encourages more returning customers while not sacrificing too much nightly profit.

It would also be wise to consider the simplicity of implementing the algorithm in a realistic setting, which would make MLFQ the least feasible algorithm. But we assume that, in this case, that is not a constraint. It also depends on the current popularity of the bar, as if there are very few customers, then FCFS performs the best, but as the popularity of the bar increases,

the effectiveness of FCFS decreases. Since we do not know the popularity of the bar, we must recommend the most consistent algorithm over all workloads.

Therefore MLFQ will be the best algorithm for Allegra the bartender as it has the best balance of algorithm efficiency, predictability, and fairness. MLQF consistently scored best or very near the best in terms of both efficiency and fairness of an algorithm. This choice will help the bar run under the best conditions where it both maximises the per-night and long-term profits of the bar.